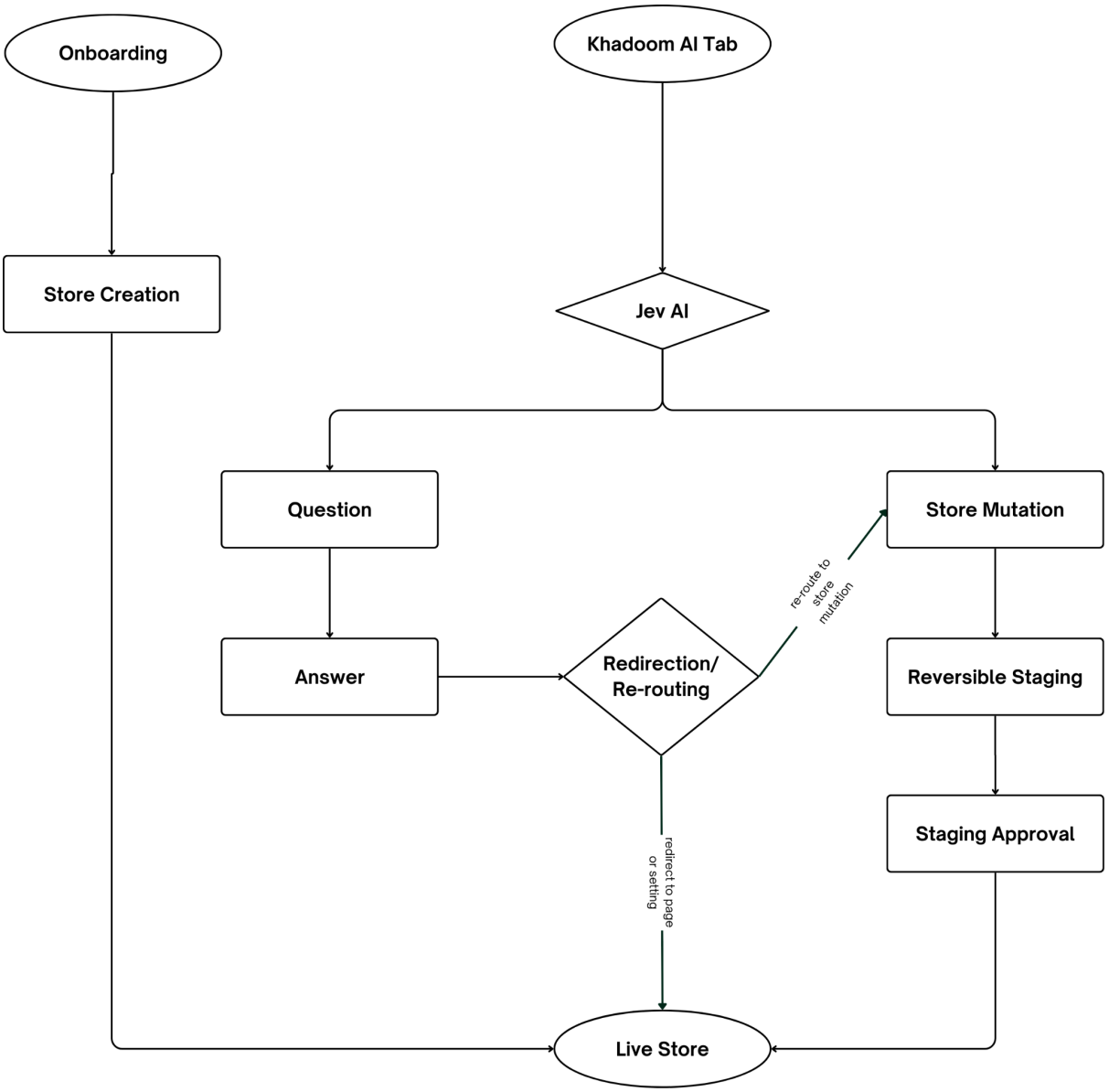
Khadoom AI: Store Creation, Mutation, And Helper



Stage 1: Onboarding & Store Creation

The user is met with a set of curated questions that guide the user into prompting the expensive LLM (GPT-4o) with everything it needs to create & customize the user's store. The LLM then updates the default schema (a complex multi-nested JSON) to match the user's answers and saves this version of the store as "v1". Finally the user is brought into the homescreen of his initial basic store setup.

Stage 2: Khadoom AI Tab

Khadoom AI tab is where the user goes after creating his store in order to further modify it or ask questions about it or about Mtajir. When the user prompts Khadoom in this tab his prompt is classified using **Jev AI** into either a question or a mutation (a complexity score is also given if the prompt is a mutation). This classification chooses which model handles the task:

Case 1 (Question): a cheap LLM (GPT-4o Mini) connected to a RAG engine handles answering questions using up-to-date documentation/data on Mtajir and the user's own store. The answer can then be either a **simple text answer**, a **redirection to a certain page/setting**, or **re-routing to the mutation pipeline** in case of questions such as: "how do i change the theme of my store to blue?", which can be handled by khadoom or manually. Also in front of the RAG engine there would be semantic caching to reduce calls to the models API for frequently repeated questions.

Case 2 (Simple & Complex Mutation): if the complexity score is low the cheap LLM model is used (GPT-4o Mini), low complexity mutations include: changing colors or description text. If the complexity score is high the expensive LLM model is used (GPT-4o), high complexity mutations include: disabling payment providers and changing shipping rates. The changes are then validated using Zod (a runtime schema validation library) and compared to the previous version, this is called the **reversible staging** phase, if any errors occur here a full retry prompt will be used and if an error occurs in some fields but not all it would trigger a mini retry of the fields/batch where the error is (these errors would be routed through GPT-4o-Mini) and then merge into the main JSON answer after fixing. And finally the **staging approval** phase is where the user is shown the before and after of the mutation (comparing the old and new versions) alongside the options to accept, reject, or modify. If changes are accepted they are saved as a new version of the **live store** for easy reversibility through version history and roll-backs.

Key Technical & Design Considerations

1. **Which LLM model should we use, and what data is required for fine-tuning or training?**

We need to test before deciding on a model, but research shows that the GPT-4o & GPT-4o-Mini models are great for the task because they are strict meaning they handle structured data well, and we can switch between the two models to decrease cost by delegating the simple tasks to the weaker Mini model. There is no need to train the models, although the question answering model should be connected to a RAG engine that contains all required data for answering any user question related to Mtajir or their store. There is also Jev AI but that isn't an LLM, rather it's a classification model that helps us choose between LLMs.

2. **What is the optimal deployment model (e.g., self-hosted private GPUs in the cloud vs. third-party APIs)?**

For the GPT models and the Jev model it has to be third-party APIs since they are not open source models meaning you can't self host them. Although as mentioned I am still looking into other models we can use, specifically open source alternatives, so again further testing is required.

3. **How will the interface bridge the gap between a fully AI-driven experience and traditional multi-level menus (similar to Shopify or Salla) for users who still prefer standard navigation?**

As for the onboarding it could either be a full transition into an AI-driven question & answer system, or keeping the normal onboarding with the addition of an input area where the user can type what he wants to do in each stage instead of interacting with the onscreen options. And as for in-app usage, the helper & mutation AI will live in a separate tab (Khadoom AI tab) and will also have an icon that can be used to interact with the AI from any tab/screen.

4. **How will we ensure Khadoom AI stays up to date?**

Staying up-to-date is mainly a requirement of the helper function which answers users' questions. The information the helper relies on is stored in a RAG engine which should be connected to a knowledge source through an updating data pipeline. When answering Mtajir-related questions the knowledge source should be the Mtajir documentation page, and when answering store-related questions the knowledge source should be the store's nested JSON which has information about everything related to each user's store independently. Another thing to watch out for is the possibility that the GPT models or the Jev model get deprecated or face an outage, so we should have backup models in place.

5. **How will we prevent hallucinations and unauthorized actions?**

As for hallucinations, Jev AI is a classification model so it cannot hallucinate text answers at all, and as for the GPT models they are strict so they are designed to respect the JSON structures but still need a process to ensure that the errors caught by Zod are handled properly, and as for the question & answer model it depends on a RAG engine which significantly reduces hallucinations. As for unauthorized actions the model only has access to what it needs.

6. **How will we protect user and business data privacy?**

Business data will not be accessed, only what we choose to reveal in the documentation behind the RAG engine is used by the model, and as for user data the model will only have access to the store's setup data so nothing private will be shared. As for the prompts themselves, according to preliminary research it does not violate privacy laws as long as it is stated that they are handled by an external API.

7. **What would the final cost look like?**

The GPT-4o costs $2.50 for input, $10 for output, and $1.25 for cached inputs. Jev AI costs $0.042 for input/output, of course all these prices are per million tokens. An estimated cost of usage per month for an active merchant with daily uses across all features would be around $15 dollars, of course with further testing that number would be more finalised.